

Justin Christopher

Enhancement Two: Algorithms and Data Structures Narrative

The artifact I selected for this enhancement is my Android Inventory App I originally developed in CS 360. The app was built to track inventory levels and demonstrate CRUD functions. It features a login screen where users can login or create an account and an inventory screen where items, quantities, and low-stock thresholds are managed. Rows highlight red when an item falls below its threshold.

Justification

I selected this artifact because it is a realistic, working Android application that touches every layer of a software system, which makes it a strong piece for all three enhancement categories. For the algorithms and data structures category, the inventory screen is where the most visible algorithmic work lives, so it was the natural focus.

The specific enhancement I made for this category was adding a live search bar and a low-stock-only filter to the inventory screen. In the original version of the application, inventory items were displayed in alphabetical order with no way to narrow the list. For a small dataset that works fine, but as inventory grows, scrolling through an unfiltered list becomes a real usability problem.

The key design decision was where to perform the filtering. The simpler approach would have been to load all items into a Java List and then loop through it to discard non-matching entries. Instead, I pushed the filtering down to the database layer by adding a new `searchItems()` method to `DBhelper` that builds a SQL WHERE clause dynamically using

a LIKE condition for the name search and a `qty <= low_threshold` condition for the low-stock filter. Both conditions can be combined at the same time. This means the database engine only returns matching rows to begin with, rather than loading the full table into memory just to throw most of it away. The `loadData()` method in `InventoryActivity` was updated to always call `searchItems()` with the current search state, and a `TextWatcher` on the search bar calls `loadData()` on every keystroke so the list updates in real time as the user types.

Course Outcomes

I planned on meeting course outcome three with this enhancement, which asks that students design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices. I believe I met this outcome. The heart of the enhancement is a deliberate trade-off analysis where filtering in Java after loading all rows is simpler to write but wastes memory and time proportional to the full table size, while filtering in SQL only returns what is needed. Choosing the more algorithmically sound approach and being able to explain why is exactly what this outcome asks for.

I do not have any updates to my coverage plan. The enhancement went as described in module one.

Reflection

The most useful thing I learned during this enhancement was how much it matters to put computation in the right layer of the system. My first instinct was to keep things simple and filter the list in Java, but thinking through what happens at scale made it clear the database was already built to do this efficiently. Redesigning `loadData()` to always call `searchItems()` regardless of whether a filter was active also ended up simplifying the code because there is only one code path instead of switching between two methods depending on state.

One challenge I ran into was handling the combination of the two filters. I had to make sure that using a search term and the low-stock checkbox at the same time did not require two separate queries or two passes over the data. The solution was to build the `WHERE` clause conditionally inside `searchItems()` based on which inputs were active, which kept all the filtering logic in one place.